

CLOUD39EV300 Platform SDK

Quick Start Guide

Chapter 1 Quick Start Guide Document Overview

This document is intended for product software development engineers to help them understand SDK-related content and quickly get started with development.

Chapter 2 Installing and Using SDK for the First Time

1. SDK package format and version description

SDK is a compressed package, generally named as: CLOUD39EV300_SDK_Vx.x.x.tgz,

Use the command:

```
tar -zxf CLOUD39EV300_SDK_Vx.x.x.tgz
```

to decompress, you can get a CLOUD39EV300_SDK_Vx.x.x directory, which is the SDK directory.

2. SDK directory introduction

CLOUD39EV300_SDK_Vx.x.x

```
|— doc
|   |— Cloud39EV300 kernel configuration and modification manual_V1.0.0.pdf
|   |— Cloud39EV300 platform user development manual_V1.0.0.pdf
|— kernel
|— platform
|   |— apps
|       |— akipc # Main application, responsible for audio and video
|       |   acquisition encoding, network transmission and local recording functions.
|       |— ccli # Dynamically obtain and set the running status of the
|       |   application process
|       |— cmd_serverd # Listen and receive shell call requests from other
|       |   processes, and initiate calls to shell commands
|       |— daemon # Background daemon, responsible for: Monitoring the
|       |   operation of anyka_ipc, and recovering from abnormal exits, memory leaks, and
|       |   kernel deadlocks. T card plug-in detection and key processing
|       |— disk_repair
|       |— Makefile
|       |— mmc_test # TF card test program
|       |— product_test # Production test program, mainly completes the testing
|       |   of hardware such as image, audio, wifi and T card, as well as completes the
|       |   burning of uid and mac addresses
|       |— updater # Upgrade program, which can upgrade boot, kernel, file
|       |   system image and upgrade package. For kernel upgrades, it is necessary to
|       |   distinguish between mixed upgrades of uImage and zImage to avoid errors
|       |— version
|   |— ChangeLog
|   |— config.mk
|   |— libapp # Application layer
```

```

├───┬─── libmpi # Multimedia layer
│   ├─── libplat # Platform layer
│   ├─── Makefile
│   └─── rootfs # File system
├─── Quick Start Guide.txt
├─── tools
├─── anyka_uclibc_gcc.tar.bz2 # Cross-compilation tool
├─── arm-anykav200-gdb-7.12.tar.gz # gdb debugging tool
└─── burnttool # Burning tool

```

3. Install and configure the cross-compilation environment on the Linux server

1. The cross-compilation tool is located in the tools directory of the SDK. The compressed package name of the tool chain provided by Ankai is anyka_uclibc_gcc.tar.bz2.

2. Use root user privileges to execute the following command in the root directory to decompress

```
sudo tar -Pxvf anyka_uclibc_gcc.tar.bz2
```

After decompression, the tool chain will be installed in the directory /opt/arm-anykav200-crosstool/. The tool chain uses an absolute path and must be placed in this directory to run normally. The directory and file names cannot be modified.

3. Set the host PATH environment variable: Use export PATH=\$PATH:/opt/arm-anykav200-crosstool/usr/bin to add it to the system's PATH variable, or add it to the system's startup script. You can modify files such as /etc/environment or /etc/bash.bashrc.

4. Toolchain confirmation: After installation, execute arm-anykav200-linux-uclibcgnueabi-gcc -v. The system can find the command and display the version information of gcc, indicating that the tool is installed correctly.

Chapter 3 Kernel and platform compilation configuration options

1. Kernel configuration options

Execute in sequence in the kernel directory:

```
mkdir ../build
make O=../build cloud39ev3_xx_defconfig
make O=../build -j4 uImage
```

Generate uImage kernel image file in the build/arch/arm/boot path.

Notes:

1. If you execute make clean directly in the kernel directory, the library files in the lib directory will be deleted. It is recommended to create a new build directory in the same directory as kernel.
2. When compiling with make O=../build -j4 uImage, the ko files of the driver modules will not be compiled. If necessary, there are several methods:
 - a. You can compile zImage directly with make O=../build -j4 without adding uImage, and compile modules at the same time, and then compile uImage with the make O=../build -j4

uImage command.

- b. Add modules to specify the target after the make O=../build -j4 uImage command, make O=../bd -j4 uImage modules.
- c. Compile separately, use make O=../bd -j4 modules to compile the driver modules, and use make O=../build -j4 uImage to compile the kernel.

2. Application cloud platform configuration

For the main application anyka_ipc, compilation configuration is required. The path of the configuration file is platform/config.mk. Due to different network configuration methods of different cloud platforms, simultaneous operation support of multiple cloud platforms is not considered for the time being, and only one cloud platform support can be enabled.

After selecting the cloud platform configuration, you must also choose whether to support Wi-Fi configuration.

The following describes the configuration items, set y to enable, and n to disable:

```
CONFIG_DANA_SUPPORT = y // Dana compilation configuration item (default)
CONFIG_RTSP_SUPPORT = n // RTSP function
CONFIG_ONVIF_SUPPORT = n // ONVIF platform
CONFIG_ONVIF_AUDIO_SUPPORT = n // Does the ONVIF platform support audio
input?
```

If the configuration in platform/config.mk is turned on, the corresponding macro definition will also be defined in the application code. For example, if CONFIG_DANA_SUPPORT = y in platform/config.mk, the macro CONFIG_DANA_SUPPORT in the application code will also be defined, and the code in the preprocessor #ifdef CONFIG_DANA_SUPPORT will be compiled.

Note: ONVIF configuration items conflict with RTSP and DANA configuration items and cannot be set to y at the same time, but RTSP configuration items do not conflict with DANA configuration items and can be y at the same time

3. File system configuration

The file system configuration file platform/rootfs/platform.cfg will automatically compile the file system of the cloud platform in the corresponding directory according to the configuration in platform/config.mk.

Configure the utils switch in platform/config.mk, which is not turned on by default. If it is not turned on, the debugging/testing tools in the rootfs/utils directory will not be copied to the rootfs.

```
CONFIG_UTILS_SUPPORT = n //utils configuration switch
```

Execute in the platform directory in sequence:

```
make clean //Clean up old compilation results
make //Compile all targets, including dynamic libraries
make install //Generate the contents of the rootfs root file system
directory, including copying the compiled application
make image //Pack the rootfs directory into an image file (root.sqsh4,
usr.sqsh4, usr.jffs2)
```

Finally, three file system image files, root.sqsh4, usr.sqsh4 and usr.jffs2, are generated in the rootfs directory

Chapter 4 System operation configuration

The system operation configuration mainly includes the following files

1. anyka_cfg.ini configuration file provides various configuration parameters for the main application
2. danale.conf configuration file saves the device login information of the Danale platform
3. isp_xx.conf and others are ISP parameter configuration files

Note: For detailed parameter configuration, please refer to "Cloud39EV300 Platform User Development Manual_V1.0.0"

Chapter 5 Install and upgrade the development board

1. Burn the image file

Note: The following content briefly introduces the burning process of the development board. For details, please refer to the "Cloud39EV200 Platform Development Board User Manual"

1. Check whether the JP4 and JP10 jumpers on the baseboard are configured as USB burning mode, that is: USB_DP->AK_DP, USB_DM->AK_DM.
2. Copy the four files uImage, root.sqsh4, usr.jffs2, and usr.sqsh4 generated by the compilation to the directory of the burning tool;
3. Run the BurnTool burning tool that comes with the PDK.
4. Use a USB cable to connect the developed USB interface to the PC USB port.
5. Press the USB power switch to turn on the power of the development board
6. Long press the [BOOT] key on the baseboard, and short press the [RESET] key on the chip core board at the same time. Release the [BOOT] key after the burning tool enters the burning mode (the corresponding channel status turns yellow).
7. In the burning tool interface, click the "Start" button to execute burning and wait for the burning to complete.
8. After the burning is successful, short press the [RESET] button on the chip board to reset and boot.

2. Development board image upgrade

Note: You can upgrade through TF card or network

1. Put the compiled uImage, root.sqsh4, usr.jffs2, usr.sqsh4 files (just put the corresponding image files as needed) into the /tmp directory on the development board through TF card or network
2. Execute the update.sh command and wait for the upgrade to complete and restart.

Chapter 6 Common development environment configuration

1. How to start using NFS file system

1. In the development stage, it is recommended to use NFS as the development environment, which can save the work of remaking and burning the root file system.
2. Operation command to mount the NFS file system:

```
mount -t nfs -o nolock -o tcp xx.xx.xx.xx:/your-nfs-path /mnt
```
3. Then you can access the files on the server in the /mnt directory and perform development work.

2. How to run and stop the main program

Note: By default, the main program is in the /usr/bin directory, and the executable file is named anyka_ipc

1. There are two ways to stop the main program:

Method 1: Use a script to stop the service. Execute the service.sh stop command to stop the running program (including anyka_ipc and daemon, etc.)

Method 2: Use the kill or killall command to kill the process directly (note that you need to kill the daemon process first, and then kill the anyka_ipc process)

2. Start the main program:

Manually starting the main program is mainly to facilitate debugging using the NFS file system. Users need to mount the program to be debugged to the development board through the NFS file system.

Then switch to the directory where the program to be debugged is located to execute the program, for example:

```
./anyka_ipc &
```

3. Enable telnet service

After the network is normal, the telnet service is generally enabled; if not, run the command `telnetd&` to start the single-board telnet service, and use telnet to log in to the single board.

4. Enable log printing

Execute `tail -F /var/log/message&` to enable log printing.